

GE HealthCare — Project Deep-Dive Drill

Vansh Rawat | EduAce + Beneath The Blue + Web Wonder

How GE asks about projects (senior + GFG)

Prepare for **any** of these on **every** resume project:

- Walk through **workflow** (user journey)
- **Backend logic** — views, models, APIs
- **Authentication & authorization** — who can access what
- **Architecture** — client, server, database (3-tier)
- **Accuracy / metrics** (if AI) — how measured, how improved
- **What you'd improve** at scale or for production
- **Testing** — unit/integration; edge cases
- **Design question** — e.g. testing software (senior report)

Tactic: Mention auth + quiz logic early to steer conversation to your strengths.

EduAce — AI-Assisted Study Platform

Stack: Python, Django, HTML/CSS | Live: <https://eduace.onrender.com/>

30-second pitch

EduAce is a web-based study assistant I built with Django. It helps students plan academic work and access AI-assisted study support in one place instead of juggling multiple tools. I owned the full stack from models and views to deployment on Render.

Workflow (be able to draw)

1. User opens site → register/login (Django auth)
2. Dashboard / study flows → user inputs goals or topics
3. Backend processes request → business logic + optional AI call
4. Response rendered in templates → user sees plan/feedback

Likely follow-ups

- **Why Django?** Batteries included — ORM, auth, admin, fast MVP for hackathon-style delivery
- **Auth flow?** User model; password hashed; session cookie; login_required on views
- **AI integration?** API call to model service; validate input before call; handle API failure gracefully
- **Accuracy?** If AI: don't claim fake metrics — say you'd add logging, user feedback thumbs up/down, guardrails on output
- **Improvements?** pytest unit tests; API rate limits; PostgreSQL instead of SQLite for prod; caching
- **Healthcare analogy?** Input validation before processing; fail-safe defaults; audit trail of user actions

Beneath The Blue — Conservation Platform

Stack: Python, Django, JS, Bootstrap | Live: <https://beneath-the-blue-1-3c6o.onrender.com/>

30-second pitch

Beneath The Blue promotes ocean conservation through an interactive Django web app. Users authenticate, take dynamically generated quizzes, and track scores. I implemented auth, quiz generation, and score tracking, and this project also became our Web Wonder finalist submission.

Architecture (3-tier — say this clearly)

- **Presentation:** HTML templates + Bootstrap + JavaScript
- **Business logic:** Django views — quiz generation, scoring rules, auth
- **Data:** Django ORM → SQLite/Postgres (User, Quiz, Score models)

Auth deep-dive (expect this)

- Registration: validate input → create User → hash password (never store plain)
- Login: verify credentials → create session
- Authorization: logged-in users access quiz; anonymous users limited pages
- Logout: destroy session

Quiz logic deep-dive

- Generate question set (from bank or rules)
- Store user answers; compute score server-side (don't trust client score)
- Edge cases: empty submission, duplicate submit, tie scores

Likely follow-ups

- **Dynamic quiz generation?** Explain data structure (question bank / random sample)
- **Security?** CSRF tokens (Django default), server-side validation, auth on POST routes
- **Improvements?** Leaderboard, admin analytics, REST API layer, mobile-responsive tests
- **Testing software design?** Unit test scoring function; integration test auth + quiz flow; CI on GitHub Actions

Web Wonder — Team Lead (Finalist)

30-second pitch

I led a team of four in Web Wonder, a competitive web development program. We delivered Beneath The Blue under deadline — I owned task breakdown, integration schedule, and the final presentation. We placed as finalists.

STAR for leadership

- **Task allocation:** Frontend vs backend vs content vs demo script
- **Risk:** Integration late — mitigated by API-first backend
- **Conflict:** Use daily standup; decide by deadline impact

Nexus — Departmental Representative

- Coordinate students and organizers for technical/cultural events
- HR angle: teamwork, communication, reliability

“Open project / live demo?”

Senior: **not guaranteed**. Still prepare:

- URLs work; test 10 min before interview
- 2-min demo script: login → one quiz → show score
- If link fails: explain architecture on whiteboard, don't panic

Coding vs logic

Senior: **mostly logic**. Still be ready to write:

- Pseudocode for quiz scoring
- Class diagram for User / Quiz / Score
- Simple Python/C++ polymorphism example